6.2.4 HOSTING

Hosting is the process of provisioning scalable server infrastructure for storing, serving, and managing web applications and relational databases accessible globally via the internet. Modern enterprise healthcare platforms require high uptime, strict data isolation, SSL/TLS cryptographic communication, and rapid continuous integration and continuous deployment (CI/CD) pipelines.

The **MediTwin AI** platform is engineered with a modern decoupled cloud architecture: the responsive client application is hosted on **Vercel's Global Edge Network**, while the 3NF-normalized PostgreSQL database and Node.js/Express API services are deployed on the **Render Cloud Application Platform**.

1. Cloud Hosting Architecture & Key Capabilities

Rather than traditional monolithic on-premise servers, utilizing specialized cloud platforms offers distinct operational advantages tailored to healthcare applications:

- **Scalable Serverless Edge Infrastructure:** Vercel distributes static assets and pre-rendered client bundles across worldwide Edge Points of Presence (PoPs), reducing latency under 50ms for clinical workstations.
- **Automated Git-Driven CI/CD Pipelines:** Every verified commit pushed to the production GitHub repository automatically initiates atomic container builds, automated regression testing, and zero-downtime canary deployments.
- **Robust Security & SSL/TLS Encryption:** Automatic issuance of TLS/SSL certificates ensures all data in transit between doctors, nurses, patients, and the database conforms to HIPAA security standards.
- **Managed PostgreSQL with Prisma ORM:** Render provisions isolated PostgreSQL container instances with automated storage scaling, continuous connection pooling, and ACID transactional guarantees.
- **Zero-Configuration Reverse Proxy Architecture:** Server-side route rewrites eliminate Cross-Origin Resource Sharing (CORS) preflight latencies and safeguard private backend service endpoints.

2. Step-by-Step Procedure for Hosting Backend on Render

Step 1: Database Schema & Prisma ORM Configuration

The backend architecture utilizes Prisma ORM to interface with PostgreSQL in strict Third Normal Form (3NF). In `backend/prisma/schema.prisma`, the PostgreSQL datasource provider and schema models (`tbl_user`, `tbl_login`, `tbl_doctor_profile`, `tbl_patient_profile`, `tbl_appointment`, `tbl_prescription`, `tbl_ward_bed`) are defined:

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}
```

Step 2: Backend Production Scripts Configuration

Ensure `backend/package.json` includes automated scripts for dependency installation, client generation, TypeScript transpilation, and database schema synchronization:

```
"scripts": {
  "build": "tsc",
  "start": "node dist/index.js",
  "prisma:generate": "prisma generate"
}
```

Step 3: Define Infrastructure as Code (render.yaml)

To ensure deterministic deployments, a declarative blueprint manifest `render.yaml` was formulated in the root repository. It instructs Render's build orchestrator to execute Prisma code generation, TypeScript compilation, and database migration:

```
services:
  - type: web
    name: meditwin-ai-backend
    env: node
    plan: free
    rootDir: backend
    buildCommand: npm install --include=dev && npx prisma generate && npm run build
    startCommand: npx prisma db push --accept-data-loss && npm start
    envVars:
      - key: NODE_ENV
        value: production
      - key: JWT_SECRET
        value: super-secret-meditwin-jwt-key
      - key: DATABASE_URL
        sync: false
```

Step 4: Push the Unified Repository to GitHub

Initialize Git, commit all workspace source files, and push the project to the centralized remote repository at `https://github.com/biyajomon11/MediTwin-AI.git` on the `main` branch.

Step 5: Sign Up and Access the Render Cloud Console

Navigate to `https://dashboard.render.com` and authenticate using authorized GitHub credentials. Grant Render read/write access to the MediTwin AI repository.

Step 6: Provision Managed PostgreSQL Database Instance on Render

1. In the Render Dashboard, click **New +** and select **PostgreSQL**.
2. Enter database name: `meditwin_db`, user: `meditwin_user`, region: `Oregon (US West)` or closest region, and PostgreSQL Version: `16`.
3. Choose the Free instance plan and click **Create Database**.

Step 7: Retrieve Internal & External Database Connection Strings

Upon database provisioning, copy the **Internal Database URL** (for zero-latency communication within Render's private virtual network) and the **External Database URL**:

```
DATABASE_URL = postgresql://meditwin_user:password@dpg-meditwin-db.oregon-postgres.render.com/meditwin_db
```

Step 8: Create and Configure Backend Web Service on Render

1. In Render Dashboard, click **New +** → **Web Service**.
2. Select the GitHub repository `biyajomon11/MediTwin-AI`.
3. Name the service: `meditwin-ai-backend`.
4. Set **Root Directory** to: `backend`.
5. Select **Environment**: `Node`; Branch: `main`.

Step 9: Configure Build Command & Start Command

Enter the exact commands matching the deployment pipeline:

- **Build Command:** `npm install --include=dev && npx prisma generate && npm run build`
- **Start Command:** `npx prisma db push --accept-data-loss && npm start`

Step 10: Inject Production Environment Variables

Under the **Environment Variables** tab of the Web Service, add the following secure keys:

- `NODE_ENV`: `production`
- `DATABASE_URL`: `postgresql://meditwin_user:password@dpg-...render.com/meditwin_db`
- `JWT_SECRET`: `super-secret-meditwin-jwt-key`
- `PORT`: `10000`
- `GOOGLE_CLIENT_ID`: `598268803778-hjlcqqdfkgrh9vld6uj9nh70k047ffbg...`

Step 11: Execute Automatic Database Schema Push & Deployment

Click **Create Web Service**. Render automatically clones the repository, installs dependencies, compiles TypeScript code to `backend/dist`, executes `prisma db push` to apply all 3NF tables to PostgreSQL, and binds the Express server to the assigned port.

Step 12: Verify Backend API Health and Live Endpoints

Once deployed, test the live backend endpoint using curl or a web browser to confirm HTTP 200 OK:

```
GET https://meditwin-ai-backend.onrender.com/api/health
Response: {"status": "healthy", "service": "MediTwin AI Backend API", "timestamp": "2026-10-21T..."}
```

3. Step-by-Step Procedure for Hosting Frontend on Vercel

Step 13: Configure Vite Production Build Pipeline

The frontend is built using React 18, TypeScript, Tailwind CSS, and Vite. Verify `vite.config.ts` has proper module aliases and production output configurations matching standard React SPAs:

```
export default defineConfig({
  plugins: [react()],
  resolve: { alias: { '@': path.resolve(__dirname, './src') } },
  server: { port: 3000 }
});
```

Step 14: Configure Single Page Application (SPA) Routing & Rewrites

Client-side routing with React Router DOM (e.g., `/dashboard/patient`, `/doctor/inpatients`, `/doctor/bed-allocation`) requires server-level fallback rewrites to `index.html` so direct link access and page reloads avoid HTTP 404 Not Found errors.

Step 15: Implement Zero-CORS API Reverse Proxy in vercel.json

To completely bypass browser Cross-Origin Resource Sharing (CORS) preflight restrictions and prevent mixed-content blocks, `vercel.json` is configured in the root directory to proxy all `/api/*` traffic server-to-server directly to the Render backend:

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "rewrites": [
    {
      "source": "/api/(.*)",
      "destination": "https://meditwin-ai-backend.onrender.com/api/$1"
    },
    {
      "source": "/(.*)",
      "destination": "/"
    }
  ]
}
```

Step 16: Configure Frontend Environment Variables

Create or update production client environment keys in `.env`:

```
VITE_GOOGLE_CLIENT_ID="598268803778-hjlcqqdfkgrh9vld6uj9nh70k047ffbg.apps.googleusercontent.com"
```

Step 17: Sign In to Vercel and Import GitHub Repository

1. Navigate to `https://vercel.com/new` and log in using GitHub OAuth.
2. Under "Import Git Repository", locate and select `biyajomon11/MediTwin-AI`.

Step 18: Configure Project Build & Output Settings on Vercel

Vercel automatically detects the Vite project structure. Verify the following parameters:

- **Framework Preset:** `Vite`
- **Root Directory:** `./`
- **Build Command:** `npm run build` (executes `tsc && vite build`)
- **Output Directory:** `dist`
- **Install Command:** `npm install`

Step 19: Add Production Environment Variables in Vercel Dashboard

Under the **Environment Variables** collapsible section, enter:

- Key: `VITE_GOOGLE_CLIENT_ID`

- Value: `598268803778-hjlcqqdfkgrh9vld6uj9nh70k047ffbg.apps.googleusercontent.com`

Step 20: Trigger Production Deployment & Global CDN Distribution

Click **Deploy**. Vercel compiles the React components, minifies CSS bundles with PostCSS, executes tree-shaking, and distributes the static application across Edge caches globally.

Step 21: Automatic SSL/TLS Certificate Provisioning

Vercel automatically assigns a globally valid SSL/TLS certificate via Let's Encrypt with HTTP Strict Transport Security (HSTS) and automatic HTTP-to-HTTPS redirect at: `https://meditwin-ai.vercel.app`.

4. Integration, Security & Post-Deployment Verification

Step 22: Configure Cross-Origin Whitelisting & Secure JWT Cookies

In `backend/src/index.ts`, configure CORS middleware to whitelist requests originating from `https://meditwin-ai.vercel.app`, enabling secure header propagation (Authorization Bearer tokens) while denying unauthorized third-party domains:

```
app.use(cors({
  origin: ["https://meditwin-ai.vercel.app", "http://localhost:3000"],
  credentials: true,
  methods: ["GET", "POST", "PUT", "DELETE", "PATCH", "OPTIONS"]
}));
```

Step 23: Perform End-to-End Clinical Workflow Smoke Tests

Conduct end-to-end integration verification across live endpoints:

- **Authentication Test:** Authenticate as Doctor, Nurse, and Patient; verify JWT issuance and session initialization.
- **Clinical Module Navigation:** Test patient prescriptions view, doctor inpatient roster, and interactive bed allocation map.
- **Data Persistence:** Record patient vitals and schedule doctor appointments; verify data commits to the live PostgreSQL database.

Step 24: Configure Automated Continuous Deployment (CI/CD) Webhooks

Vercel and Render deploy automatic webhooks listening to the GitHub repository. Every push to the `main` branch triggers concurrent pipeline builds:

- **Vercel Pipeline:** Clones updated commit → runs `npm run build` → updates Edge CDN in ~35 seconds.
- **Render Pipeline:** Re-pulls backend → executes `prisma generate` → applies schema migrations → restarts container with zero downtime.

Step 25: Real-Time Server Monitoring, Log Streaming & Maintenance

Live application metrics, memory limits, CPU utilization, and HTTP request access logs are tracked in real time via the Render Log Viewer and Vercel Analytics console. Automated daily backups ensure database disaster recovery.

Hosted Application Deployment Summary

Deployment Parameter	Production Value / Platform Specification
Frontend Platform	Vercel Global Edge Network (Vite / React 18 / Tailwind CSS)
Live Frontend URL	**https://meditwin-ai.vercel.app**
Backend API Platform	Render Cloud Application Services (Node.js / Express / Prisma)
Live Backend API Endpoint	https://meditwin-ai-backend.onrender.com
Database Infrastructure	Render Managed PostgreSQL 16 (3NF Relational Architecture)
Source Code Repository	GitHub: `biyajomon11/MediTwin-AI` (Branch: `main`)
Security & Protocols	End-to-End HTTPS (TLS 1.3), JWT Auth, Reverse Proxy Rewrites
Deployment Pipeline	Automated CI/CD via GitHub Push Webhooks



Scan to Launch MediTwin AI Web Application

https://meditwin-ai.vercel.app